



Анджали Гопинадхан Наир

Безопасность пакетов Python в 2026 году: как атаки на цепочки поставок нацелены на вашу среду разработки ИИ

Мнение

7 августа 2026 года • 5 минут

Ваши разработчики доверяют своим инструментам. Это доверие используется не по назначению.



24 марта 2026 года разработчики, создававшие ИИ-приложения с помощью [LiteLLM](https://www.litellm.ai/) [https://www.litellm.ai/] — пакета Python, который скачивают 95 миллионов раз в месяц, — по незнанию установили вредоносный код. Группа злоумышленников, известная как TeamPCP, взломала конвейер распространения PyPI и добавила в индекс пакетов вредоносные версии 1.82.7 и 1.82.8. Вредоносная программа была малозаметной: это был файл .pth, малоизвестный механизм Python, который автоматически запускает код при каждом запуске интерпретатора. Если вы установили одну из скомпрометированных версий, вредоносный код запускался автоматически — явный импорт не требовался.

Это уже не исключение. Это закономерность.

Что происходит на самом деле

По данным [ReversingLabs](https://www.reversinglabs.com/blog/software-supply-chain-security-report) [https://www.reversinglabs.com/blog/software-supply-chain-security-report], в 2026 году количество вредоносных пакетов с открытым исходным кодом выросло на 73%. Атака на LiteLLM была частью более масштабной кампании TeamPCP, в ходе которой злоумышленники систематически взламывали широко распространённые инструменты безопасности с открытым исходным кодом, в том числе Trivy от Aqua Security и KICS от Checkmarx, а затем перешли к библиотекам инфраструктуры ИИ, размещённым на PyPI. Цепочка атак на LiteLLM развивалась по уже знакомой схеме. TeamPCP получила учетные данные администратора PyPI, опубликовала вредоносные версии, которые были практически неотличимы от официального пакета, и внедрила многоэтапную полезную нагрузку, предназначенную для сбора ценных секретов: токенов AWS, GCP и Azure, ключей SSH и учетных данных облачных сервисов. По данным [Zscaler ThreatLabz](https://www.zscaler.com/blogs/security-research/supply-chain-attacks-surge-march-2026) [https://www.zscaler.com/blogs/security-research/supply-chain-attacks-surge-march-2026], зараженные пакеты были доступны в течение примерно трех часов, после чего их поместили в карантин. Трех часов было достаточно, чтобы проникнуть в десятки тысяч корпоративных систем.

И это коснулось не только LiteLLM. В конце апреля 2026 года в версиях PyTorch Lightning 2.6.2 и 2.6.3 было обнаружено вредоносное ПО для кражи учетных данных, которое запускалось при импорте. Один вредоносный файл рабочего процесса раскрывал секреты во всех конвейерах CI/CD.

Почему среды разработки ИИ особенно уязвимы

Большинство атак на цепочки поставок — это плохо. Атаки на цепочки поставок, нацеленные на среды разработки ИИ, — еще хуже.

Среды искусственного интеллекта и машинного обучения объединяют разработку, исследования, облачную инфраструктуру, доступ к данным, публикацию моделей и автоматизацию в рамках одного рабочего пространства. Взломанный пакет Python в стандартном веб-приложении может привести к краже учетных данных базы данных. Та же атака в среде разработки ИИ может привести к раскрытию весов модели, обучающих данных, облачных токенов от нескольких провайдеров, секретов конвейера CI/CD и ключей рабочих API — и все это одновременно из-за одной зараженной зависимости.

Есть еще один уровень, который большинство специалистов по безопасности не учитывают. Когда разработчики используют ИИ-помощников для написания кода, те часто предлагают директивы `pip install` и операторы импорта, которые ссылаются на определенные пакеты. Если разработчик доверяет предложению и устанавливает указанный пакет, а злоумышленник уже зарегистрировал вредоносный пакет под этим именем, атака будет успешной, даже если злоумышленник не взаимодействует с разработчиком напрямую. Исследователи назвали это «слэпсквоттингом». Недавнее исследование [<https://arxiv.org/pdf/2605.17062>] показало, что из почти 200 000 запросов на Python все основные большие языковые модели генерируют вымышленные имена пакетов, которых нет на PyPI, создавая постоянную уязвимость, которую не может устранить ни одно обновление модели.

Your developers are not doing anything wrong. They are using the tools that make them productive. The security assumption underneath those tools is broken.

3 controls that matter right now

1. Pin your dependencies and verify integrity

Floating version specifiers — `requests>=2.0` rather than `requests==2.31.0` — allow package managers to silently pull updates that include malicious code. Pin every dependency in your AI development environments to an exact version and verify checksums against a known-good hash. This alone would have limited the blast radius of the LiteLLM attack to environments that explicitly upgraded to the compromised versions rather than any environment that ran `pip install litellm` without constraints.

2. Audit post-install hooks in your development pipeline

The LiteLLM attack embedded its payload using Python's `.pth` file mechanism — code that executes automatically during interpreter initialization, before any import statement runs. Post-install hooks and `.pth` file manipulation are a documented attack class, but enforcement in developer environments is inconsistent. Require review of packages that include post-install scripts before they reach developer machines. Tools like Socket and Sonatype provide real-time analysis of PyPI packages for malicious behavior before

installation. This is not a nice-to-have. Given the pace of AI tooling adoption, it is a basic control.

3. Rotate cloud credentials immediately after any suspected exposure

The LiteLLM payload targeted AWS, GCP and Azure tokens specifically because those credentials provide lateral movement across cloud environments. If your development pipelines pulled LiteLLM during the March 24 exposure window, treat every cloud credential accessible from those environments as potentially compromised and rotate them. Review your cloud provider audit logs for activity patterns that do not correspond to developer-initiated requests — the signature of a stolen token being used by an attacker in a different location.

What this means for security teams

The TeamPCP campaign is not the end of this pattern. It is a proof of concept that AI infrastructure is now a target class. LiteLLM, PyTorch Lightning and the tools in between are packages your AI teams depend on every day. The attackers know that. They know that developers move fast, that AI tooling adoption outpaces security review cycles and that a malicious .pth file is invisible to most endpoint detection products.

The controls above are not complex. They do not require new vendors or new platforms. They require treating Python package installation in AI development environments with the same rigor you apply to production deployments — because in 2026, the distance between a developer’s local environment and your production infrastructure is shorter than it has ever been, and attackers have noticed.

Your developers trust their tools. Make sure that trust is warranted.

Python[https://www.csoonline.com/python/]	Programming Languages[https://www.csoonline.com/programming-languages/]
Software Development[https://www.csoonline.com/software-development/]	Code Security[https://www.csoonline.com/code-security/]
Security[https://www.csoonline.com/security/]	

About



Policies



More



Our Network

